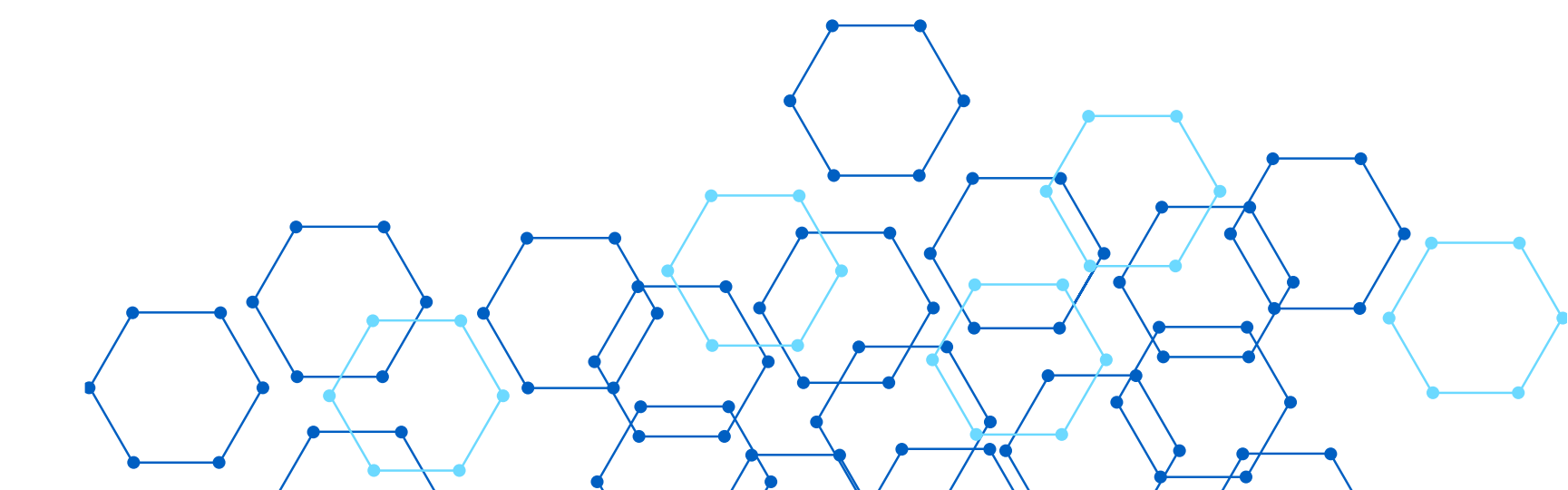
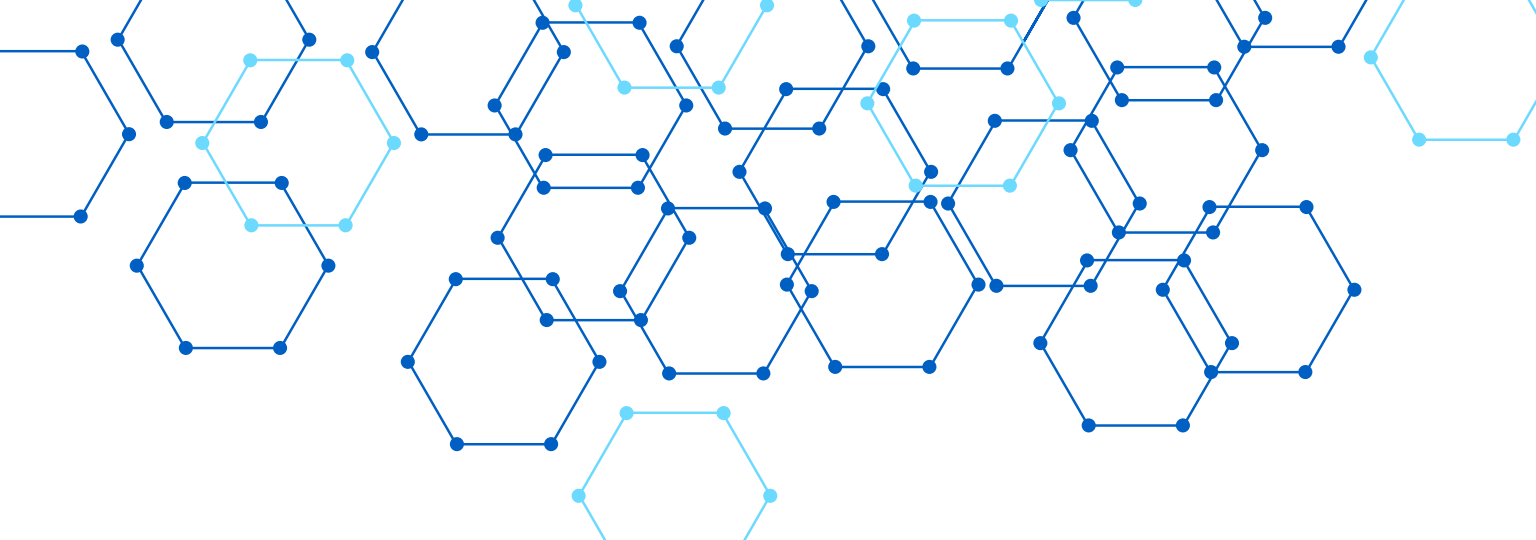




ARCHITECTURE LOGICIELLE ET PATRONS EN GÉNIE LOGICIEL

PLANT

1/ Architecture Logique VS Physique

2/ Critères de qualité de logiciels liées à l'architecture
(conception architecturale)

2/ Définition d'un patron en GL

3/ Types de Patrons

4/ Patrons d'architectures : MVC - 3 Couches - 5 couches

5/ Patrons de conception : 3 catégorie - 23 patrons

6/ Détailler 2 patrons de conception de chaque catégorie



INTRODUCTION À L'ARCHITECTURE LOGICIELLE

L'architecture logicielle définit la structure fondamentale d'un système informatique : ses composants, leurs relations et les principes de conception qui les régissent. Elle influence directement la qualité, la maintenabilité et l'évolutivité du logiciel. Une bonne architecture facilite la performance, la sécurité, la scalabilité et organise efficacement la séparation des responsabilités pour un travail d'équipe optimal.

QU'EST-CE QUE L'ARCHITECTURE LOGIQUE ?



L'architecture logique représente l'organisation fonctionnelle d'un système logiciel, indépendamment des aspects techniques d'implémentation. Elle se concentre sur le « QUOI » : quels modules composent le système et comment ils interagissent. Les composants sont regroupés par domaine métier (gestion utilisateurs, commandes, catalogue) pour une meilleure cohérence fonctionnelle.



Les relations entre modules sont définies via des interfaces, API et flux de données clairement spécifiés. Par exemple, une application e-commerce comprend des modules Catalogue, Panier, Paiement et Utilisateur, chacun avec ses responsabilités propres. Cette séparation facilite la compréhension, la maintenance et l'évolution du système.

QU'EST-CE QUE L'ARCHITECTURE PHYSIQUE ?



L'architecture physique définit le déploiement réel du logiciel sur l'infrastructure matérielle. Elle se concentre sur le « COMMENT » et le « OÙ » : serveurs web, serveurs d'application, bases de données, systèmes de cache, configuration réseau et dispositifs de sécurité.



Exemple concret : une application e-commerce déployée sur 2 serveurs web frontaux, 3 serveurs d'application pour la logique métier, un cluster MySQL pour la persistance des données, un serveur Redis pour le cache, et un Load Balancer pour la répartition de charge.

ARCHITECTURE LOGIQUE VS PHYSIQUE

Définition

Logique : Organisation fonctionnelle indépendante de l'infrastructure

Physique : Déploiement réel sur matériel et réseau

Acteurs

Logique : Architectes, développeurs

Physique : DevOps, administrateurs système



Objectif

Logique : Organiser les fonctionnalités et modules métier

Physique : Optimiser le déploiement et les performances

Niveau d'abstraction

Logique : Élevé (conceptuel)

Physique : Bas (concret)

Focus

Logique : Le « QUOI » - structure fonctionnelle

Physique : Le « OÙ » et « COMMENT » - infrastructure

Outils

Logique : UML, diagrammes de composants

Physique : Diagrammes de déploiement, monitoring



CRITÈRES DE QUALITÉ



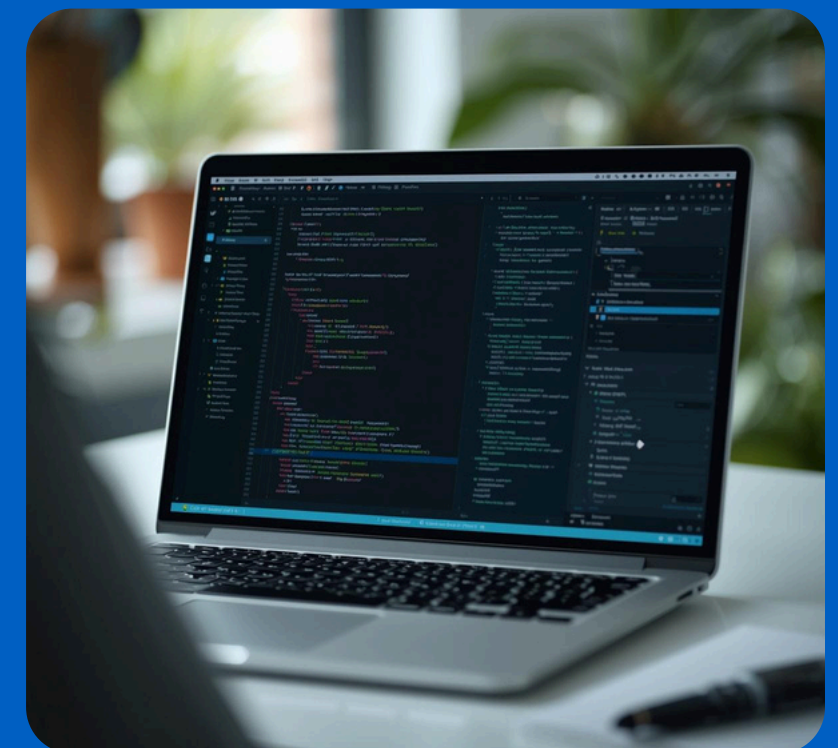
Maintenabilité : Simplicité d'évolution et de correction du code.
Une architecture bien conçue permet des modifications rapides sans effets secondaires indésirables.



Performance & Sécurité : Efficacité du temps de réponse et optimisation des ressources. Protection des données sensibles et gestion rigoureuse des accès utilisateurs.



Fiabilité, Testabilité & Scalabilité : Tolérance aux erreurs et pannes. Facilité des tests unitaires et système. Gestion efficace de la montée en charge et réutilisabilité des composants.



DÉFINITION D'UN PATRON (DESIGN PATTERN)

Un patron de conception est une solution réutilisable à un problème récurrent en conception logicielle. Ce n'est pas du code prêt-à-l'emploi, mais une approche adaptable selon le contexte.

Avantages clés :

- Vocabulaire commun entre développeurs
- Capitalisation de l'expérience collective
- Gain de temps considérable
- Meilleure organisation du code
- Maintenance facilitée
- Application des bonnes pratiques éprouvées



LES TYPES DE PATRONS EN GÉNIE LOGICIEL



Patrons d'architecture : Structure globale du système (ex : MVC, microservices, architecture en couches). Définissent l'organisation générale et les interactions entre composants majeurs.



Patrons de conception : Solutions au niveau classes/objets (ex : Singleton, Factory, Observer). Résolvent des problèmes récurrents de conception orientée objet.



Patrons de programmation (idiomes) : Techniques spécifiques au langage utilisé. Bonnes pratiques d'implémentation propres à Java, Python, C++, etc.



Model

The diagram illustrates the MVC (Model-View-Controller) pattern. It consists of three main components: Model, View, and Controller. The Model is represented by a blue rounded rectangle at the top. The View is represented by a blue rounded rectangle in the middle. The Controller is represented by an orange rounded rectangle at the bottom. Arrows indicate the flow of data: a downward arrow from Model to View, and another downward arrow from View to Controller. On the left side, there are decorative blue lines and dots. On the right side, there is a network-like structure of blue lines and dots.

View

Controller

PATRON MVC (MODEL-VIEW-CONTROLLER)

Model : Gère les données et la logique métier. Stocke l'état de l'application, traite les règles métier et communique avec la base de données. Indépendant de l'interface utilisateur.

View : Affiche les données à l'utilisateur. Responsable de l'interface graphique, reçoit les données du Model via le Controller. Exemples : pages HTML, composants React.

ARCHITECTURE 3 COUCHES



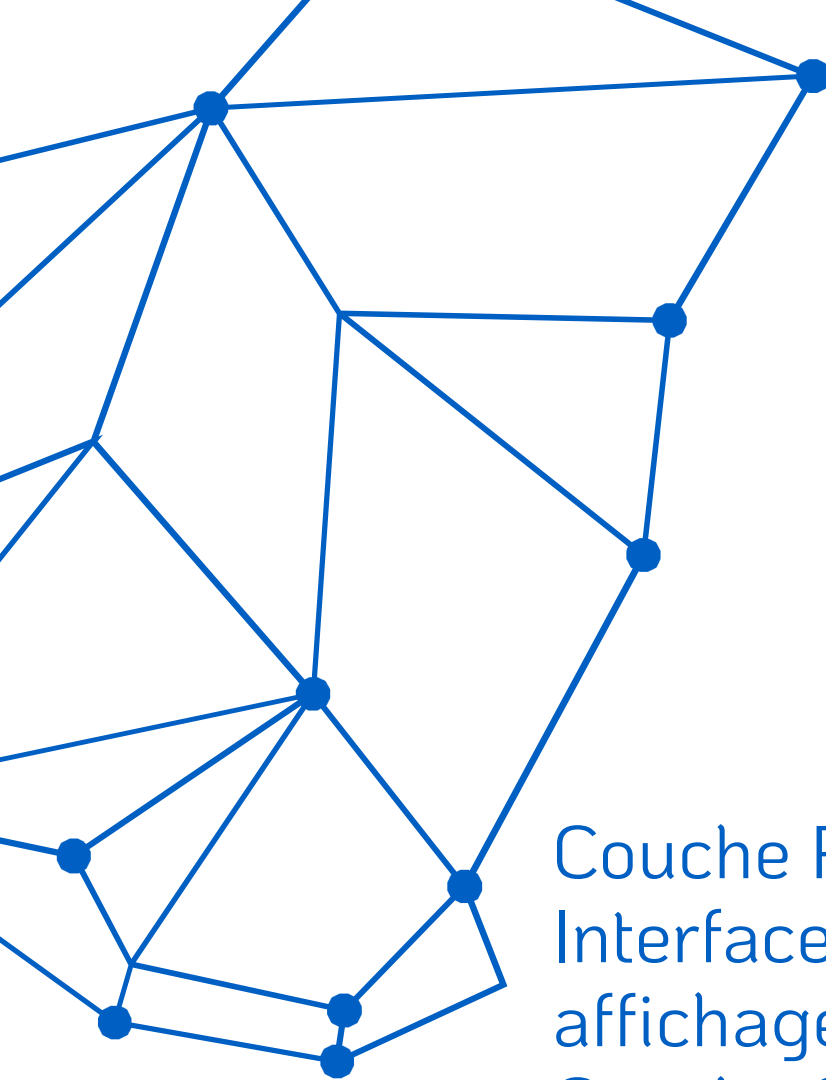
Couche Présentation : Interface utilisateur (UI/UX), affichage des données, gestion des interactions utilisateur. Communication avec la couche métier via interfaces définies.



Couche Métier : Logique applicative et règles de gestion, traitement des données, orchestration des processus. Fait le lien entre présentation et accès aux données.



Couche Accès aux Données : Interaction avec les bases de données, requêtes SQL/NoSQL, persistance. Avantages : modularité, maintenance facilitée, testabilité accrue.



ARCHITECTURE 5 COUCHES

Couche Présentation
Interface utilisateur et
affichage des informations.
Couche Service
Orchestration et exposition
des API métier.
Couche Métier
Logique applicative et règles
de gestion.



Extension de l'architecture
3 couches avec granularité
accrue. Les 5 couches
permettent une modularité
renforcée, facilitent
l'architecture distribuée et
améliorent la scalabilité et
l'intégration des systèmes
complexes.

Couche Accès aux Données
Interaction avec les bases de
données et systèmes de
stockage.
Couche Données
Stockage persistant des
informations (SQL, NoSQL,
fichiers).



Software Design Patterns

This creations al Software v creagtiabes the poftrms consicnity costery
cleebate deeravmetres ware your enartive phy withite and biatfnes
the consteence comereciield-nfrograne.



Structiral
Patterns



Creation
Patterns



Behavioral
Patterns

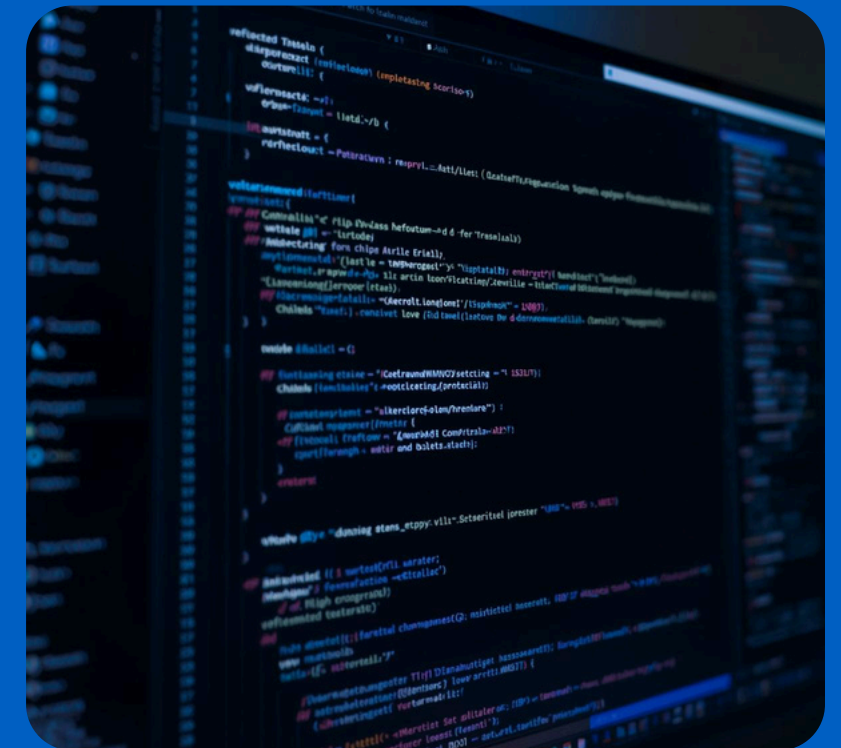
PATRONS DE CONCEPTION GOF : INTRODUCTION

Le Gang of Four (GoF) a publié en 1994 un ouvrage fondateur présentant 23 patrons de conception classés en trois catégories : création, structure et comportement. Ces patrons constituent un vocabulaire commun pour les développeurs et offrent des solutions éprouvées aux problèmes récurrents de conception logicielle.

EXEMPLES CLÉS DE PATRONS GOF



Singleton : Garantit une instance unique globale d'une classe. Cas d'usage : gestionnaire de configuration, connexion base de données, logger système.



Factory : Délègue la création d'objets aux sous-classes. Cas d'usage : création de documents (PDF, Word), génération d'interfaces selon plateforme.



Observer, Strategy, Decorator : Observer notifie les dépendants (événements UI). Strategy permet d'interchanger algorithmes (tri, paiement). Decorator ajoute des responsabilités dynamiquement (composants graphiques).

PATRONS D'ARCHITECTURE MODERNES

Microservices

Services indépendants avec bases de données dédiées. Chaque service gère un domaine métier spécifique.

Avantages : déploiement indépendant, scalabilité ciblée, technologies variées, résilience accrue.



Architecture hexagonale

Séparation logique métier et techniques via ports/adaptateurs. Le cœur métier reste isolé des détails d'implémentation externes.

CQRS

Command Query Responsibility Segregation : séparation lecture/écriture pour optimiser performances et scalabilité.



Architecture événementielle

Communication par événements asynchrones entre services découplés. Idéale pour IoT, systèmes temps réel et intégration.

Serverless

Exécution sans gestion serveur (AWS Lambda, Azure Functions). Facturation à l'usage, scalabilité automatique.

BONNES PRATIQUES ARCHITECTURALES



Principes SOLID : Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Fondements d'un code maintenable et extensible.

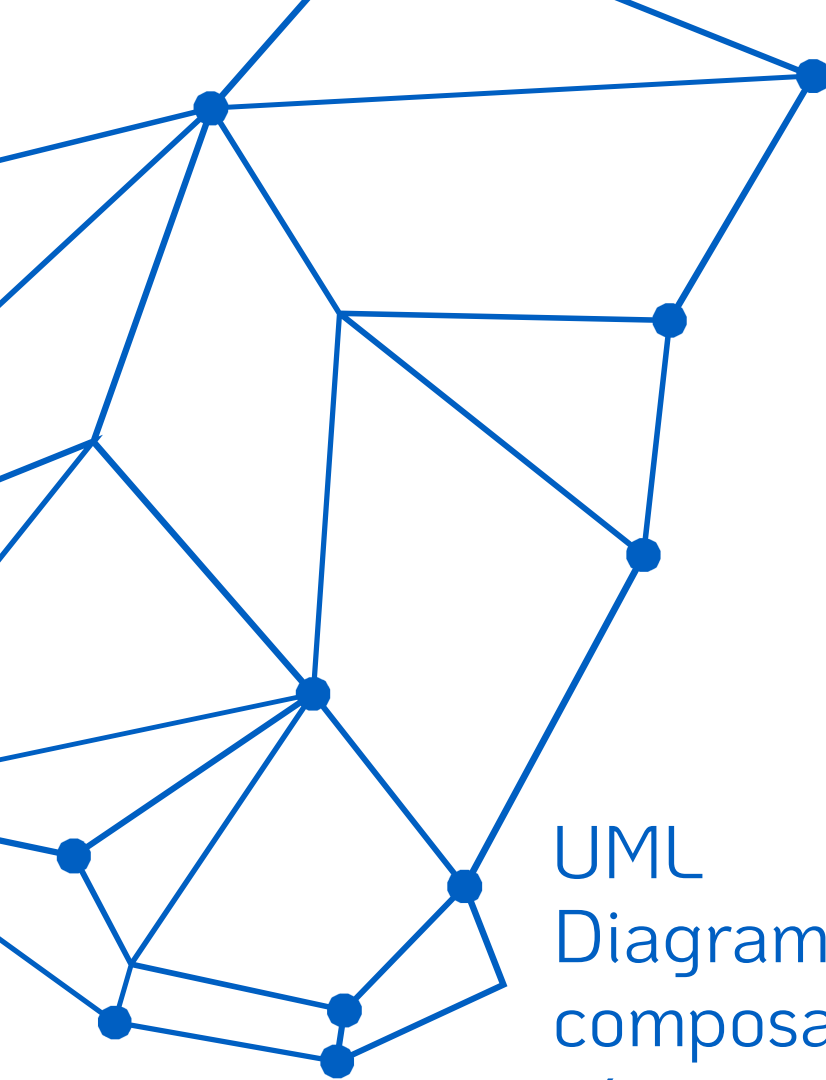


Couplage faible et cohésion forte : modules indépendants avec responsabilités bien définies. Principe DRY (Don't Repeat Yourself) pour éviter la duplication du code.



Documentation essentielle : diagrammes UML, ADR pour tracer les décisions. Tests à tous les niveaux : unitaires, intégration, système. Revues régulières de l'architecture.

OUTILS ET MÉTHODES DE MODÉLISATION



UML

Diagrammes de classes, composants, déploiement et séquence. Standard universel de modélisation. ADR pour tracer les décisions architecturales.



ArchiMate
Modélisation d'architecture d'entreprise complète. Visualisation des couches métier, application et technologie. Standard ouvert pour documentation globale.

C4 Model

Documentation à 4 niveaux : Contexte, Containers, Composants, Code. Approche progressive et claire. Outils : Lucidchart, Draw.io, PlantUML, Miro.



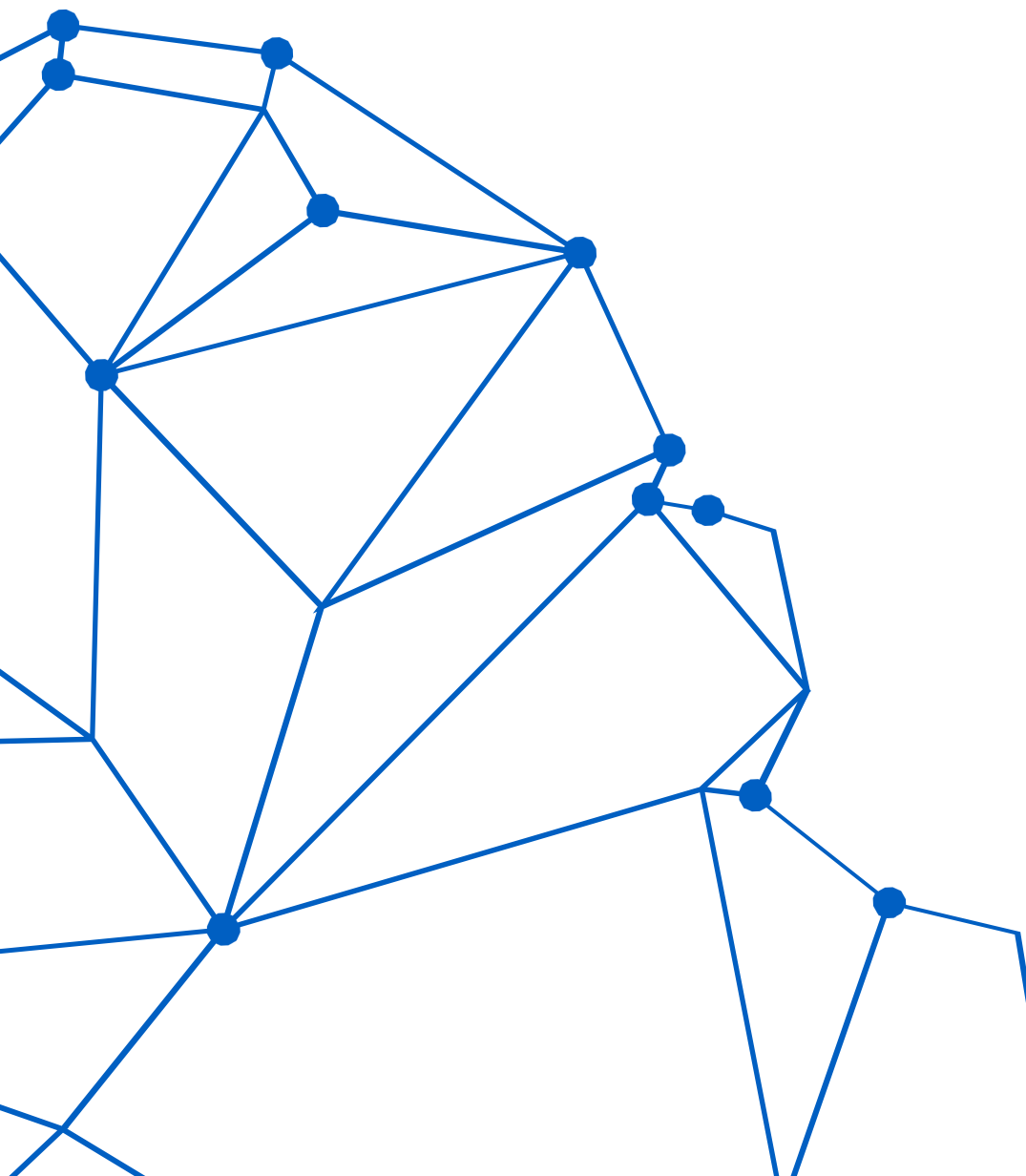
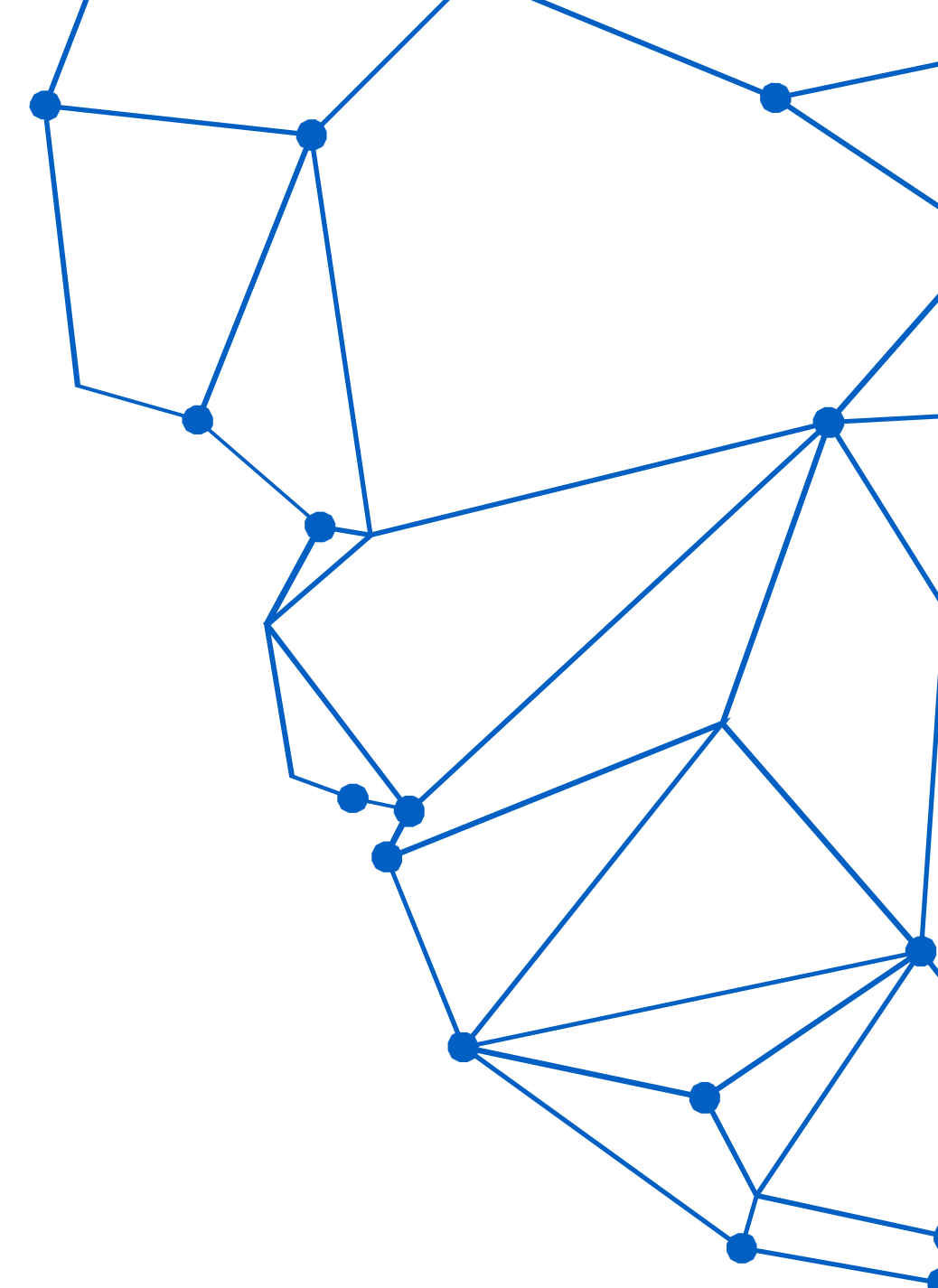
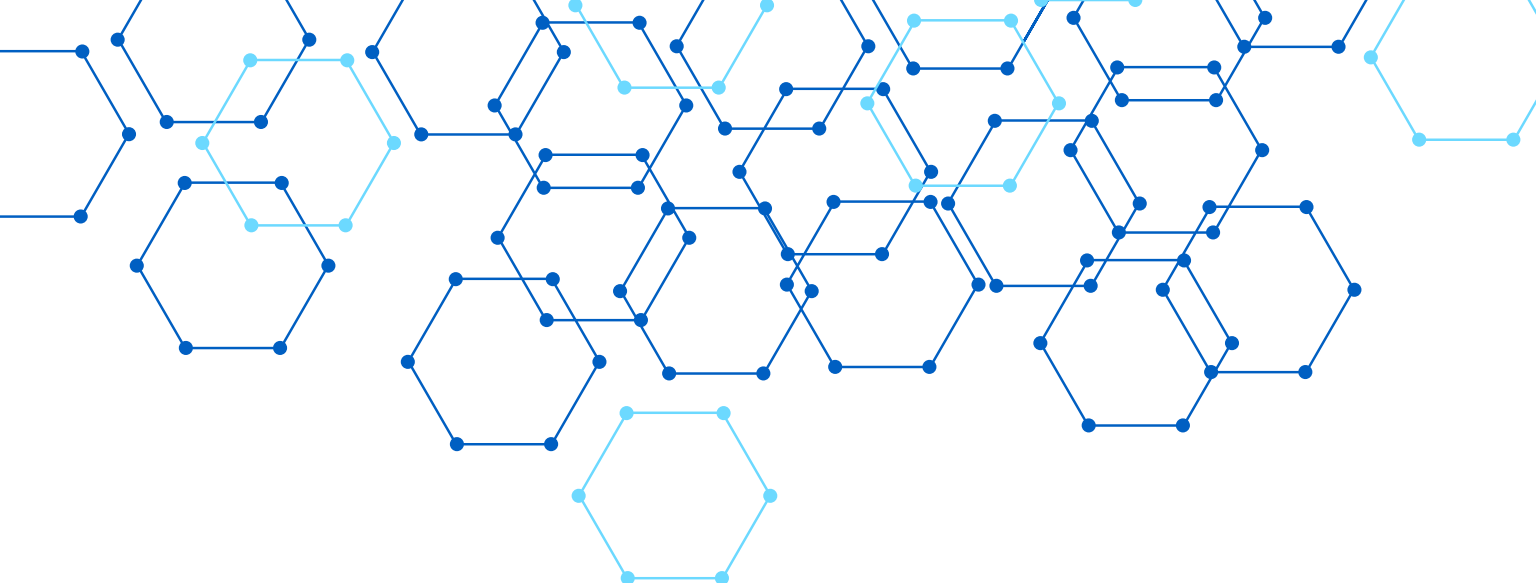
DÉFIS ET ÉVOLUTION DE L'ARCHITECTURE LOGICIELLE



Dette technique : Accumulation de choix techniques sous-optimaux nécessitant une gestion proactive. Complexité croissante : L'adoption du cloud et des microservices multiplie les points de défaillance et la difficulté de monitoring. Sécurité dès la conception : Intégration des pratiques DevSecOps et protection contre les menaces émergentes.



Performance et scalabilité : Anticiper la montée en charge dès les phases initiales de conception. Principe YAGNI : Trouver l'équilibre entre flexibilité architecturale et simplicité, en évitant la sur-ingénierie. Évolution des besoins : Adapter continuellement l'architecture aux nouvelles exigences fonctionnelles et techniques.



MERCI

